Package ‘loco’

June 29, 2026

Type Package

Title Functions to Calculate and Plot Local Correlations Between Features in High-Dimensional Point-Cloud Data

Version 0.1.0

Author Tim Stohn

Maintainer Tim Stohn <tim.stohn@yahoo.com>

Description Quickly detect local correlation patterns between features in point cloud data (like single-cell data in biological applications). Instead of clustering the data points and looking for correlations within every cluster
LoCo creates neighbourhoods of points, detects correlations within those neighbourhoods and filters correlations that change smoothly through the point cloud.

License GPL-3 + file LICENSE

LinkingTo Rcpp

Imports Rcpp,ggplot2,uwot,stats

SystemRequirements C++17

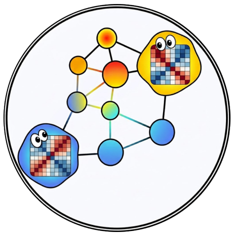
Encoding UTF-8

RoxygenNote 7.1.2

R topics documented:

loco-package	*loco: Local Correlation Detection in High-Dimensional Point-Cloud Data*

Description



LoCo (Local Correlations) detects feature pairs whose correlation changes systematically across a high-dimensional point cloud – for example, gene pairs that are co-expressed only in a specific sub-population of cells in single-cell RNA-seq data.

Algorithm overview

LoCo works in three stages:

1. **Neighbourhood construction** – Random anchor cells are sampled and a KNN graph is built to collect a fixed number of nearest neighbours around each anchor. Each anchor plus its neighbours forms one *neighbourhood*.
2. **Local correlation** – Within every neighbourhood, pairwise Spearman (or Pearson) correlations are computed for all feature pairs (or a user-supplied subset). Only correlations above `correlationCutoff` that appear in at least `corrSetAbundance` fraction of neighbourhoods are retained.
3. **Laplacian scoring** – The retained correlations are scored with a Laplacian smoothness criterion. A low Laplacian score indicates that the correlation is spatially coherent (i.e., it changes smoothly and systematically across the point cloud rather than appearing randomly). A permutation test yields p-values for each scored pair.

Typical workflow

```
library(loco)

# 1. Run LoCo on your data matrix (cells x features TSV)
result <- run_loco("my_data.tsv", correlationCutoff = 0.4,
                   neighbourhoodSize = 50)

# 2. Inspect the top-scoring (most spatially coherent) pairs
head(result$LaplacianScores)

# 3. Embed into UMAP space for visualisation
result <- add_umap_coords(result)

# 4. Plot a specific correlation pair across the UMAP
p <- plot_local_correlation_map(result, result$LaplacianScores$FeaturePair[1])

# 5. Zoom into a spatial region to see cell-level scatter
p2 <- plot_cell_level_correlation(result,
```

```
featureA = "GeneA", featureB = "GeneB",
x_min = -3, x_max = 0, y_min = -2, y_max = 2)
```

Functions

- `run_loco` Main entry point – runs the full LoCo pipeline and returns neighbourhoods, per-neighbourhood correlations, and Laplacian-scored feature pairs.
- `add_umap_coords` Computes a UMAP embedding of the raw data and attaches UMAP coordinates to the result list so that correlations can be plotted in 2-D space.
- `plot_local_correlation_map` Plots a chosen feature pair's per-neighbourhood correlation strength as a colour-coded scatter across any 2-D embedding.
- `plot_cell_level_correlation` Plots a cell-level scatter of two features, restricted to cells that fall inside a user-defined spatial window.

`add_umap_coords` *Add UMAP coordinates to a LoCo result*

Description

Computes a UMAP embedding of the raw data returned by `run_loco` and attaches the 2-D coordinates (`UMAP1`, `UMAP2`) to the `RawData`, `Neighbourhoods`, and `Correlations` data frames in the result list. The enriched list can then be passed directly to `plot_local_correlation_map` or `plot_cell_level_correlation`.

Usage

```
add_umap_coords(
  locoResult,
  n_pcs = 0,
  scale = TRUE,
  metric = "euclidean",
  seed = 7
)
```

Arguments

`locoResult`	Named list returned by `run_loco`.
`n_pcs`	Integer. Number of principal components to compute before running UMAP. Default `0` skips PCA and runs UMAP directly on the feature matrix. Set to a positive integer (e.g. `30`) to first reduce dimensionality with PCA – recommended for datasets with many features, as it speeds up UMAP and often improves the embedding quality. When `n_pcs` > `0`, `scale` is automatically set to `FALSE` because PCA already centres and scales the data.
`scale`	Logical. If `TRUE` (default), z-score normalise the feature matrix before computing UMAP. Ignored when `n_pcs` > `0`.
`metric`	Character string. Distance metric passed to `umap`. Default: `"euclidean"`. Other options supported by **uwot** (e.g. `"cosine"`) are also accepted.
`seed`	Integer. Random seed for reproducible UMAP layouts. Default: `7`.

Details

To visualise where in the point cloud a correlation is strong or weak, LoCo needs a 2-D layout. This function provides one by running UMAP on the feature matrix stored in `locoResult$RawData`. Each neighbourhood is then assigned the UMAP coordinates of its anchor cell, so that neighbourhood-level statistics (e.g. per-neighbourhood correlation strength) can be plotted in the same 2-D space as the individual cells.

You are not required to use this function. If you have an existing embedding (PCA, t-SNE, spatial coordinates, ...), you can add columns `UMAP1` and `UMAP2` (or any name of your choice) to `locoResult$Correlations` and `locoResult$Neighbourhoods` manually and pass custom dimension names via the `dim1`/`dim2` arguments of the plot functions.

`plot_cell_level_correlation` *Plot cell-level correlation within a spatial window*

Description

Produces a cell-level scatter plot of `featureA` vs `featureB` using only the cells that belong to neighbourhoods whose anchor falls inside the specified 2-D bounding box (`x_min`–`x_max`, `y_min`–`y_max`).

This is the companion to `plot_local_correlation_map`: first use the map to visually identify a region of the embedding where a pair shows strong (or weak) correlation, then use this function to inspect the underlying cell-level relationship in that region.

Usage

```
plot_cell_level_correlation(
  locoResult,
  featureA,
  featureB,
  x_min,
  x_max,
  y_min,
  y_max,
  dim1 = "UMAP1",
  dim2 = "UMAP2"
)
```

Arguments

`locoResult`	Named list returned by `run_loco` after embedding coordinates have been added (e.g. via `add_umap_coords`).
`featureA`	Character string. Name of the first feature (plotted on the x-axis of the scatter). Must be a column in `locoResult$RawData`.
`featureB`	Character string. Name of the second feature (plotted on the y-axis of the scatter). Must be a column in `locoResult$RawData`.

x_min	Numeric. Lower bound of the x-axis filter applied to neighbourhood anchor positions. Neighbourhoods whose anchor has a dim1 coordinate **below** this value are excluded.
x_max	Numeric. Upper bound of the x-axis filter. Neighbourhoods whose anchor has a dim1 coordinate **above** this value are excluded.
y_min	Numeric. Lower bound of the y-axis filter applied to neighbourhood anchor positions. Neighbourhoods whose anchor has a dim2 coordinate **below** this value are excluded.
y_max	Numeric. Upper bound of the y-axis filter. Neighbourhoods whose anchor has a dim2 coordinate **above** this value are excluded.
dim1	Character string. Column in locoResult$Neighbourhoods used as the x-axis coordinate for spatial filtering. Default: "UMAP1".
dim2	Character string. Column in locoResult$Neighbourhoods used as the y-axis coordinate for spatial filtering. Default: "UMAP2".

Value

A ggplot2 scatter plot of featureA vs featureB for all cells in the selected spatial window.

---

plot_local_correlation_map *Plot per-neighbourhood correlation strength across an embedding*

Description

Visualises how the correlation of a chosen feature pair varies spatially across the point cloud. Each neighbourhood is drawn as a point at the position of its anchor cell in the 2-D embedding (e.g. UMAP), coloured by the Spearman/Pearson correlation of the pair inside that neighbourhood. Blue indicates negative correlation, white indicates no correlation, and red indicates positive correlation.

Run add_umap_coords first to attach UMAP coordinates, then call this function:

```
result2 <- add_umap_coords(result)
plot_local_correlation_map(result2, result2$LaplacianScores$FeaturePair[1])
```

Usage

```
plot_local_correlation_map(
  locoResult,
  correlationPair,
  dim1 = "UMAP1",
  dim2 = "UMAP2"
)
```

Arguments

- `locoResult` — Named list returned by `run_loco` after embedding coordinates have been added (e.g. via `add_umap_coords`). The `Correlations` data frame must contain the columns named by `dim1` and `dim2`.
- `correlationPair` — Character string. Name of the feature pair to plot, formatted as `"featureA_featureB"`. Valid names are listed in the `FeaturePair` column of `locoResult$LaplacianScores`.
- `dim1` — Character string. Name of the column in `locoResult$Correlations` to use as the x-axis. Default: `"UMAP1"` (set by `add_umap_coords`).
- `dim2` — Character string. Name of the column in `locoResult$Correlations` to use as the y-axis. Default: `"UMAP2"` (set by `add_umap_coords`).

Value

A `ggplot2` object. Save with `ggplot2::ggsave()`.

---

run_loco — *Run LoCo analysis*

Description

Runs the complete LoCo (Local Correlations) pipeline on a feature matrix. LoCo samples random anchor cells, builds fixed-size neighbourhoods around them, computes pairwise feature correlations within each neighbourhood, and scores the resulting correlation pairs with a Laplacian smoothness criterion. Pairs with a low Laplacian score vary systematically across the point cloud (e.g. a gene pair that is tightly co-expressed only in one cell sub-population) rather than appearing randomly. A permutation test gives each pair a p-value.

Usage

```
run_loco(
  inFile,
  del = "\t",
  col = FALSE,
  row = FALSE,
  zscore = TRUE,
  thread = 1,
  correlatedSetMode = 1,
  numberCorrelations = 0,
  cellStateGeneFile = "",
  correlationStateGeneFile = "",
  numberNeighbourhoods = 0,
  neighbourhoodSize = 100,
  neighbourhoodKNN = 5,
  correlationCutoff = 0.5,
  permutations = 100,
```

```
    minSetSize = 2,
    corrSetAbundance = 0.01,
    correlationType = "spearman",
    calcFeatureSets = FALSE
)
```

Arguments

- `inFile` Path to the input data file. The file must be a delimited text matrix where rows are cells and columns are features (genes, proteins, ...), or vice versa – see `col` and `row`.
- `del` Single character used as the column delimiter in `inFile`. Default: `"\t"` (tab-separated).
- `col` Logical. If `TRUE`, feature names are in the *columns* (i.e. each row is a cell and each column is a feature – the standard layout). If `FALSE` (default), LoCo treats the file as transposed (rows = features, columns = cells).
- `row` Logical. If `TRUE`, cell (row) names are present as the first column of the file and will be read as cell IDs. If `FALSE` (default), LoCo assigns automatic IDs of the form `C_<index>` starting from 0.
- `zscore` Logical. If `TRUE` (default), each feature is z-score normalised (zero mean, unit variance) before correlation is computed. Recommended for data with very different dynamic ranges across features.
- `thread` Integer. Number of parallel threads to use. Default: `1`. Increase for large datasets to speed up neighbourhood and correlation calculations.
- `correlatedSetMode`

  Integer controlling which graph algorithm is used to group co-correlated features into *feature sets*. Only relevant when `calcFeatureSets = TRUE`. Accepted values:

  `0` **– Clique / fully-connected** Finds maximal cliques using the Bron-Kerbosch algorithm. Every feature in a returned set is directly correlated with every other feature in that set. Produces the most stringent, compact sets but is computationally expensive for large graphs.

  `1` **– Connected components (default)** Groups features into weakly connected subgraphs. Two features end up in the same set if they are linked through any chain of pairwise correlations, even if not all pairs are directly correlated. Fast and suitable for most use cases.

  `>= 2` **– Minimum-edge connectivity** Each feature in a returned set must share at least `correlatedSetMode` direct correlations with other members of that set. For example, `correlatedSetMode = 3` requires every feature to have at least 3 correlated partners within the set. Higher values yield denser, more tightly interconnected sets.
- `numberCorrelations`

  Integer. Maximum number of feature pairs to evaluate. `0` (default) computes all $n(n-1)/2$ pairwise correlations. Set to a positive integer to subsample pairs, which is useful for very large feature spaces where exhaustive computation would be prohibitive.

`cellStateGeneFile`
: Path to a plain-text file (one feature name per line, no header) listing the features to be used *exclusively* for neighbourhood construction. Only these features drive the KNN graph that defines which cells are neighbours. Useful when a curated gene set (e.g. cell-type marker genes) should define cell proximity independently of the features being correlated. Leave as `""` (default) to use all features for neighbourhood construction.

`correlationStateGeneFile`
: Path to a plain-text file (one feature name per line, no header) listing the features for which pairwise correlations are computed. All other features are ignored during the correlation step. Leave as `""` (default) to compute correlations for all features.

`numberNeighbourhoods`
: Integer. Total number of neighbourhoods to create. `0` (default) lets LoCo automatically choose $n_{\text{cells}}/\text{neighbourhoodSize}$ neighbourhoods so that the point cloud is covered roughly once. Increase for denser coverage or set explicitly to cap compute time.

`neighbourhoodSize`
: Integer. Number of cells in each neighbourhood (anchor cell + its nearest neighbours). Default: `100`. Larger neighbourhoods capture broader, more global co-expression patterns; smaller neighbourhoods capture finer local variation. The neighbourhood size must be meaningfully smaller than the total number of cells.

`neighbourhoodKNN`
: Integer. Number of nearest neighbours used to build the KNN graph from which neighbourhoods are grown. Default: `5`. This controls the graph connectivity used when expanding from the anchor cell to fill the neighbourhood to `neighbourhoodSize` cells.

`correlationCutoff`
: Numeric in $[0, 1]$. Minimum absolute correlation required for a feature pair to be recorded within a neighbourhood. Default: `0.5`. Pairs whose correlation falls below this threshold in a given neighbourhood are discarded for that neighbourhood. Lower values retain weaker correlations; higher values restrict to strongly correlated pairs only.

`permutations`
: Integer. Number of permutations used to estimate the null distribution for the Laplacian score p-value. Default: `100`. Increase (e.g. to 1000) for more accurate p-values.

`minSetSize`
: Integer. Minimum number of features required to report a feature set when `calcFeatureSets = TRUE`. Sets smaller than this threshold are discarded. Default: `2`.

`corrSetAbundance`
: Numeric in $[0, 1]$. Minimum fraction of neighbourhoods in which a feature pair must show a correlation above `correlationCutoff` to be considered for Laplacian scoring. Default: `0.01` (1 %). Increase to focus on correlations that appear consistently across the dataset; decrease to also capture very rare, localised correlations.

`correlationType`
: Character string, either `"spearman"` (default) or `"pearson"`. Type of correlation computed within each neighbourhood. Spearman correlation is rank-based and

more robust to outliers; Pearson correlation is faster but sensitive to non-linear relationships and extreme values.

calcFeatureSets
: Logical. If TRUE, LoCo additionally groups co-correlated features into *feature sets* (modules) using the algorithm selected by correlatedSetMode. A feature-level graph is built across all neighbourhoods: an edge is drawn between two features if their pair passes the corrSetAbundance and correlationCutoff filters globally (not restricted to a single spatial region). The resulting feature sets are reported in the FeatureSet column of LaplacianScores. Default: FALSE.

Details

Pipeline steps executed by run_loco:

1. The input matrix is parsed and, if requested, z-score normalised.
2. numberNeighbourhoods anchor cells are sampled at random. A KNN graph (k = neighbourhoodKNN) is built from the cell coordinates (using all features or only those in cellStateGeneFile), and each anchor cell collects its neighbourhoodSize nearest neighbours to form a neighbourhood.
3. Within each neighbourhood, pairwise correlations (correlationType) are computed for all feature pairs (or the subset in correlationStateGeneFile). Pairs below correlationCutoff are discarded for that neighbourhood.
4. Pairs that survive the corrSetAbundance filter (present in at least that fraction of all neighbourhoods) are scored with the Laplacian smoothness criterion and given a permutation-based p-value.
5. Optionally (calcFeatureSets = TRUE), features are grouped into co-correlated modules via the graph algorithm chosen by correlatedSetMode.

Interpreting the Laplacian score: A *low* score means the correlation is spatially smooth – it varies gradually and systematically across the point cloud, suggesting a biologically meaningful local co-regulation. A high score means the correlation is noisy or random.

Value

A named list containing LoCo results with four elements:

RawData A data.frame containing the input expression matrix in long form, with an additional column cellID. If the raw data did not contain named cellIDs the new cellIDs are enumerated in the form C_<index>, where the first index starts from 0. Rows correspond to cells and columns correspond to features.

LaplacianScores A data.frame summarising statistically significant feature pairs based on Laplacian scoring. Contains:

- FeaturePair: names of feature pairs as feature1_feature2
- LaplacianScore: computed laplacian score
- p_value: permutation-based p-value
- FeatureSet: comma-separated list of features forming sets of co-correlated features

Correlations A data.frame of correlation values per neighbourhood with 3 columns "CorrelationPair", "NeighbourhoodID", "Correlation". The first column CorrelationPair contains feature pairs (same name as in LaplacianScore), the second column NeighbourhoodID contains all neighbourhoodIDs (like N_<number>), and the third column Correlation contains the correlation of the pair in this neighbourhood.

Neighbourhoods A data.frame describing all constructed neighbourhoods:

- NeighborhoodID: unique ID of each neighbourhood in the form N_<index> where the index starts from 0.
- AnchorCellID: central cell of the neighbourhood. LoCo starts by sampling random cells (anchor cells) and builds local neighbourhoods around them.
- AllCellIDs: comma-separated list of all cells in this neighbourhood. These were the closest cells to the anchor cell.

Examples

```
# Load example dataset shipped with the package
infile <- system.file("example", "data_1.tsv", package = "loco")

# Run LoCo on a small example dataset: find all local correlations with a correlation above 0.4
L1 <- run_loco(infile, correlationCutoff= 0.4, neighbourhoodSize = 25)

# Inspect the correlations with the lowest laplacian score / p-value -
# these correlations change the most across the whole single-cell dataset but change only very little in their local
head(L1$LaplacianScores)
L2 <- add_umap_coords(L1)
plot <- plot_local_correlation_map(L2, L2$LaplacianScores$FeaturePair[1])
ggplot2::ggsave(filename = "local_correlation_map.png",
plot = plot,
width = 8,
height = 6,
dpi = 300)
```